Task 1: Stride Pattern Prefetching

Task 1A: IP-stride prefetcher (prefetcher/ip_stride.l1d_pref)

The IP-stride prefetcher learns, for each load instruction (IP), the distance between the cache lines it touches one after another. If the same distance (stride) shows up twice in a row, it prefetches the next few lines at that stride.

Table

It keeps a table of 24 entries (IP_TRACKER_COUNT). Each entry has:

- ip: the load IP, only its low 8 bits (ip & MASK_IP, MASK_IP = 0xFF). Different IPs with the same low 8 bits share an entry.
- last_cl_addr: the last cache line address this IP accessed (address >> 6, since a line is 64 bytes).
- last_stride: the stride between the last two lines this IP accessed, in cache lines (signed).
- lru: the LRU position, 0 = most recently used, 23 = least recently used.

The table is fully associative: an IP can be in any of the 24 entries, and a lookup compares the IP with all of them. At start-up the lru fields are set to 0..23 so that every entry has a distinct position.

When it runs

ChampSim calls l1d_prefetcher_operate(addr, ip, cache_hit, type, ...) on every demand load that reaches the L1D, both hits and misses. Stores do not train it.

Step by step

1. Compute the cache line address: cl_addr = addr >> LOG2_BLOCK_SIZE, and mask the IP to 8 bits.
2. Search all 24 entries for this IP.
3. If the IP is not found (a new IP):
    - pick the entry whose lru is 23 (the least recently used one) and overwrite it: new ip, last_cl_addr = cl_addr, last_stride = 0;
    - make it the most recently used: every entry with a smaller lru value moves down by one, and this entry gets lru = 0;
    - return without prefetching. A new IP has no history yet.

4. If the IP is found, compute the stride cl_addr - last_cl_addr in cache lines. The code subtracts the smaller value from the larger and then fixes the sign, because the addresses are unsigned and a direct subtraction would wrap around for a negative stride.
5. If the stride is 0 (same line again), return. Nothing is updated, not even LRU.
6. If the stride equals last_stride (the same stride twice in a row), issue PREFETCH_DEGREE = 3 prefetches, to lines cl_addr + stride*1, cl_addr + stride*2, cl_addr + stride*3:
    - Page check: each prefetch address must be in the same 4 KB page as the current access (LOG2_PAGE_SIZE = 12). At the first address outside the page, the loop stops.
    - Fill level: if the L1D MSHR is less than half full (fewer than 8 of 16 entries in use), the prefetch is sent with FILL_L1 and is brought into the L1D. Otherwise it is sent with FILL_L2 and is brought only into L2. This keeps prefetches from using up the L1D MSHRs that demand misses need.
7. Store the new state: last_cl_addr = cl_addr, last_stride = stride, and make the entry most recently used (same LRU update as in step 3).

Example

An IP loads lines A, A+2, A+4, A+6, ...

- A: new IP, entry allocated, no prefetch.
- A+2: stride 2, last stride 0, no prefetch. last_stride = 2.
- A+4: stride 2 = last stride, prefetch A+6, A+8, A+10.
- A+6: stride 2 again, prefetch A+8, A+10, A+12 (A+8 and A+10 were already prefetched, so they are not fetched again).

Limitations

- It only catches a constant stride. A repeating pattern such as (+1, +2, +2, +2, +3) breaks it: the stride must repeat twice in a row before it prefetches, and it then predicts the old stride again even when the pattern is about to change.
- Its confidence is just "same stride twice", so one repeat is enough to start prefetching.
- The degree is fixed at 3. It never backs off when its prefetches are useless.
- With only 24 entries and an 8-bit tag, IPs evict each other often when a program has many load instructions.

Task 1B: Stride Pattern Prefetcher (prefetcher/spp.l1d_pref)

Design

SPP starts from the IP-stride code and adds a pattern path and throttling.

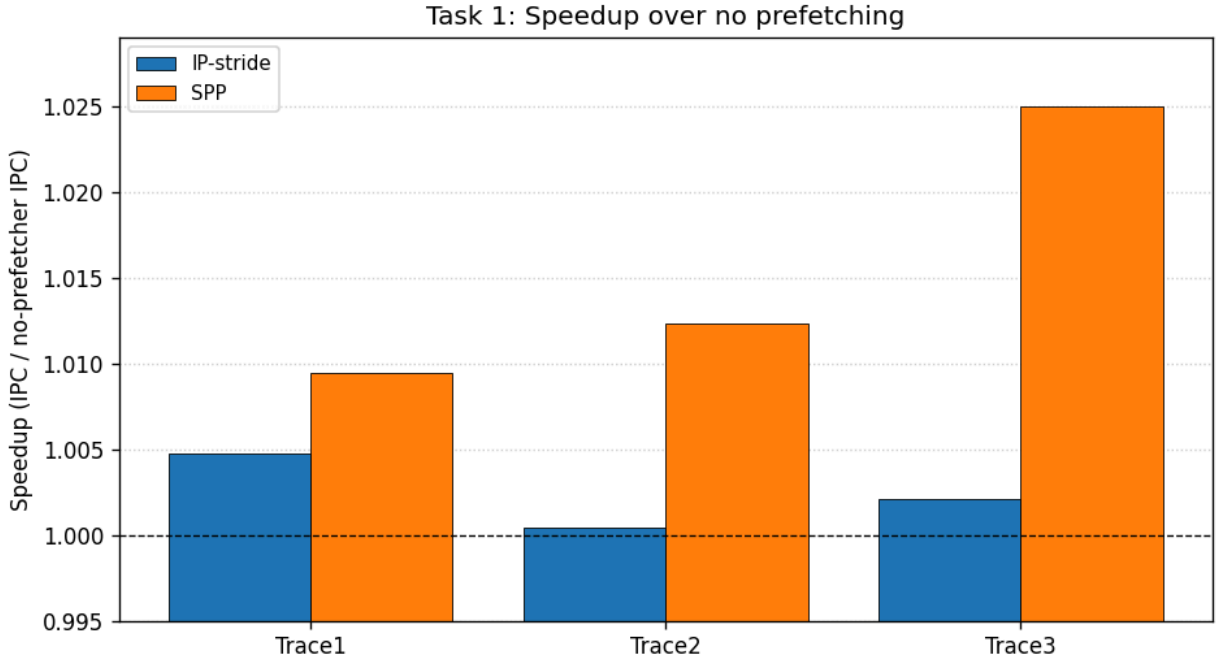
- IP table: 64 entries, fully associative, LRU, 8-bit IP tag, as in IP-stride but larger. Each entry stores the last line address (low 16 bits), the last stride, a 2-bit uniform confidence conf and a 12-bit signature sig.
- Uniform strides: conf is incremented when the new stride equals the previous stride and reset to 0 when it differs, so conf >= 2 means the same stride was seen 3 times in a row. Then SPP prefetches cl_addr + k*stride for k = 1..degree, like IP-stride. This path is checked first, so a uniform stride has priority.
- Non-uniform strides: sig holds the last 3 strides of the IP, 4 bits each (strides -8..+7). A shared 4096-entry pattern table, indexed by sig, stores the stride that followed that signature and a 2-bit confidence. On every access (if the signature is not empty and the new stride fits in 4 bits) the entry for the old signature is trained with the stride that actually came next: same stride as stored, confidence + 1; different stride, confidence - 1; different stride at confidence 0, the entry is replaced with the new stride at confidence 1. If the IP is not in uniform mode, SPP looks up the current signature, predicts the next stride, adds it to the address, shifts it into a copy of the signature and looks up again (lookahead), up to the degree. The walk stops at an entry with confidence 0. A stride outside -8..+7 clears the signature.
- Example: For the pattern (+1, +2, +2, +2, +3) repeating, the five signatures each map to the correct next stride, so SPP predicts the whole pattern.
- Page boundary: in both paths, a prefetch outside the 4 KB page of the current access stops the loop.
- Fill level: same MSHR rule as IP-stride (L1D if the MSHR is less than half full, else L2).
- Throttling: uniform and pattern mode each have their own degree, chosen from 1, 2, 4, 8. Every prefetch carries a tag saying which mode issued it. Each mode's accuracy is (useful prefetches) / (prefetches filled into L1D). After every 64 of a mode's prefetches have filled the L1D, its degree moves one level: up if accuracy >= 60%, down if accuracy < 40%, otherwise unchanged. Both modes start at degree 4.

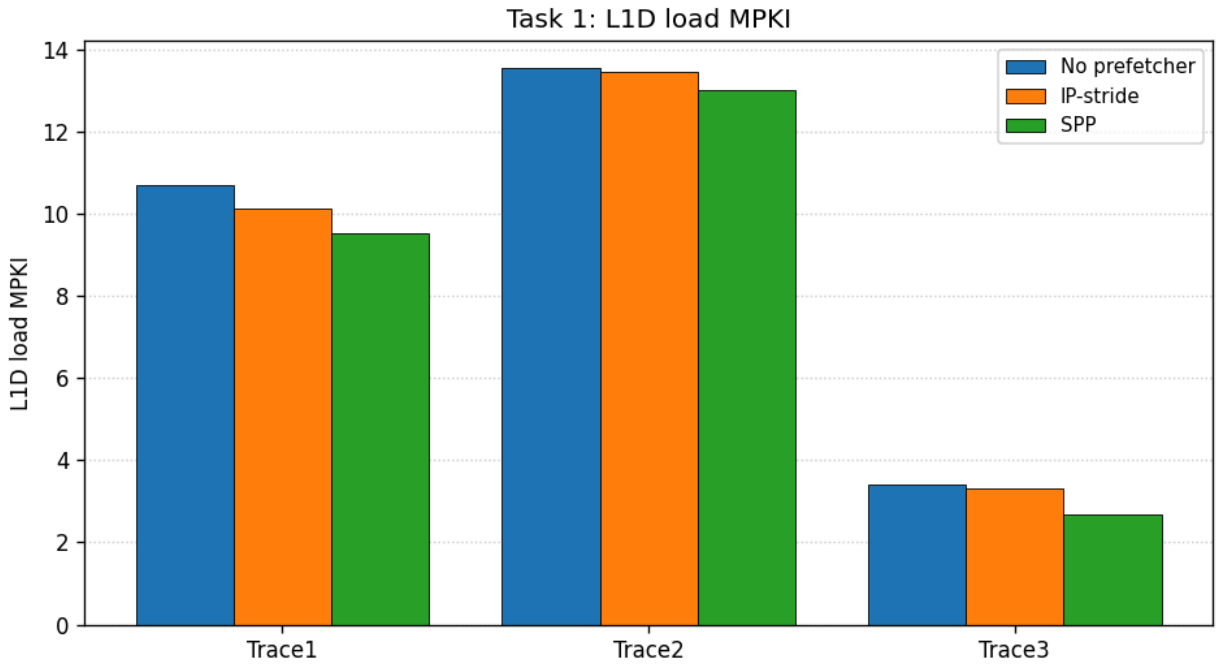
Results

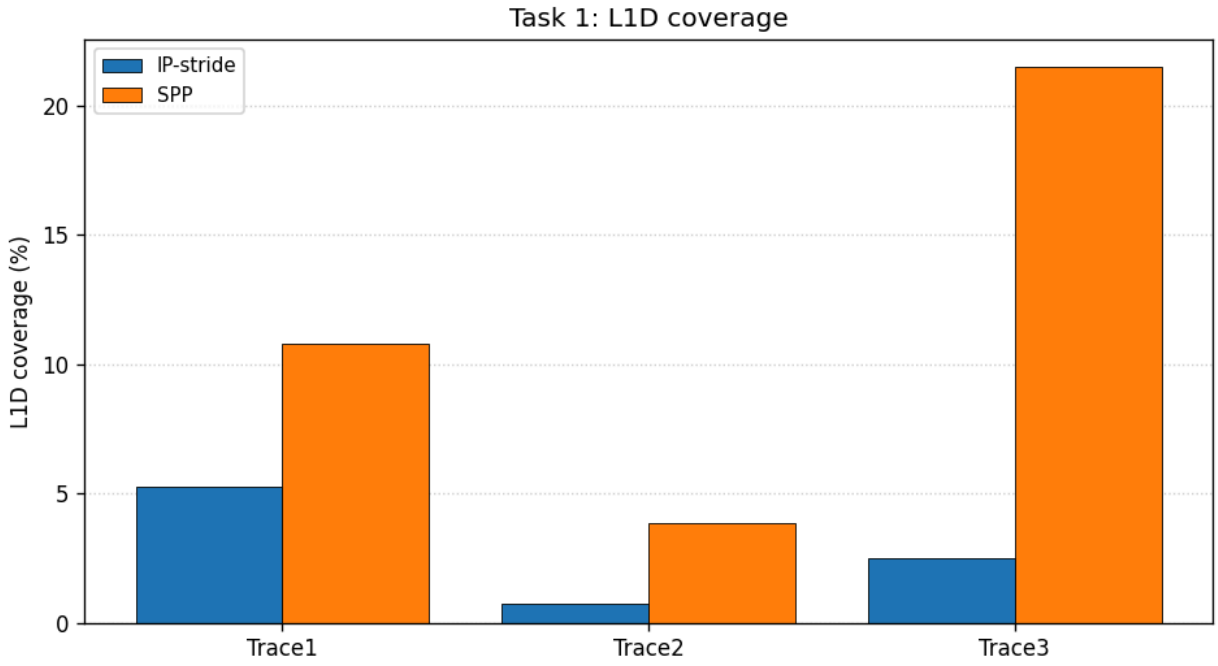
Speedup = IPC / IPC with no prefetcher. Accuracy is ChampSim's L1D accuracy (useful prefetches / prefetches issued to the lower level). Coverage = (baseline L1D load misses - L1D load misses) / baseline L1D load misses.

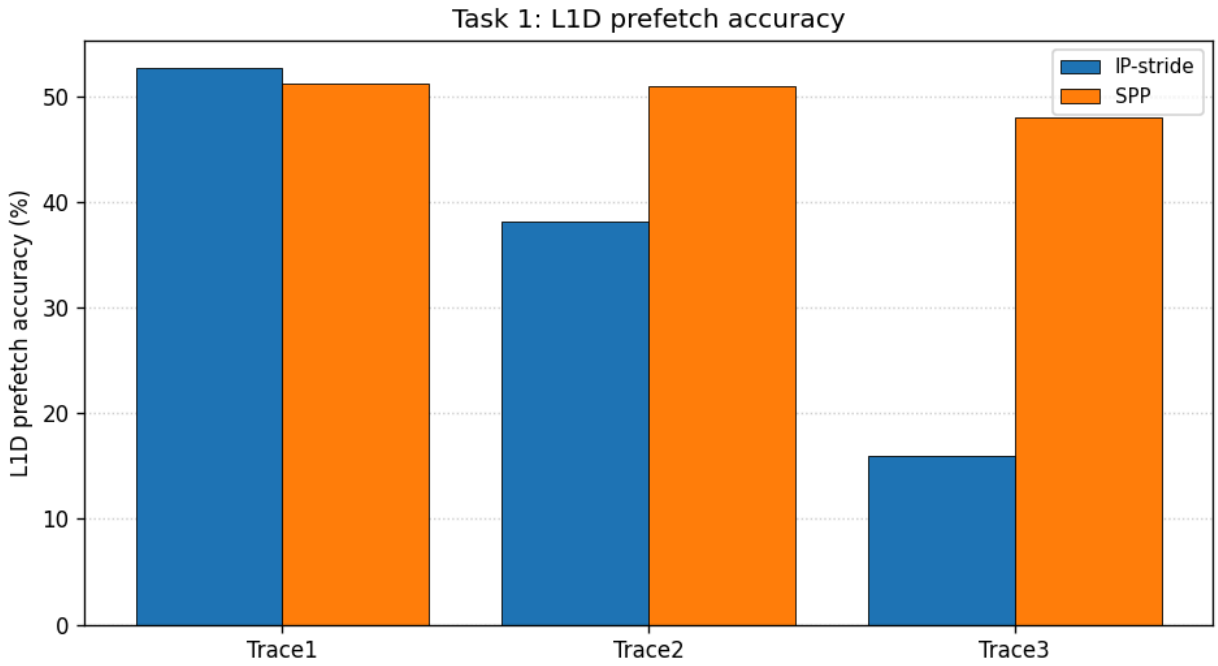
Trace	Prefetcher	IPC	Speedup	L1D Load MPKI	Accuracy	Coverage
Trace1	No prefetcher	0.232345	1.0000	10.678	-	-
Trace1	IP-stride	0.233452	1.0048	10.119	52.7%	5.2%
Trace1	SPP	0.234546	1.0095	9.528	51.3%	10.8%
Trace2	No prefetcher	0.251158	1.0000	13.538	-	-
Trace2	IP-stride	0.251260	1.0004	13.438	38.2%	0.7%
Trace2	SPP	0.254259	1.0123	13.018	51.1%	3.8%
Trace3	No prefetcher	0.382065	1.0000	3.403	-	-
Trace3	IP-stride	0.382862	1.0021	3.318	16.0%	2.5%
Trace3	SPP	0.391621	1.0250	2.672	48.0%	21.5%

Geomean speedup: IP-stride 1.0024, SPP 1.0156.


Task 1: Speedup over no prefetching
IP-stride
SPP
Speedup (IPC / no-prefetcher IPC)
1.025
1.020
1.015
1.010
1.005
1.000
0.995
Trace1
Trace2
Trace3


Task 1: L1D load MPKI
No prefetcher
IP-stride
SPP
L1D load MPKI
14
12
10
8
6
4
2
0
Trace1
Trace2
Trace3





SPP is faster than IP-stride on all three traces and removes 2-9 times as many L1D load misses. Its accuracy is about the same as IP-stride on Trace1 and much higher on Trace2 and Trace3, so the extra coverage does not come from prefetching more blindly. The gains are small in absolute terms because most L1D misses in these traces are not stride-predictable.

Storage overhead

Hardware bit widths (not C++ variable widths).

Structure	Field	Bits	Reason
IP table entry	ip tag	8	low 8 bits of the IP
	last_cl_addr	16	low 16 bits of the line address; stride = sign-extended 16-bit difference
	last_stride	16	range of a 16-bit difference
	lru	6	position 0..63 in a 64-entry fully associative table
	conf	2	uniform confidence, saturates at 3
	sig	12	last 3 strides x 4 bits
IP table	64 entries x 60 bits	3,840	
Pattern table entry	next_stride	4	stride -8..+7
	conf	2	confidence, saturates at 3
Pattern table	4096 entries x 6 bits	24,576	
Throttle (per mode)	degree level	2	4 levels: 1/2/4/8
	epoch fill counter	7	counts to 64
	epoch useful counter	7	counts to 64 (saturating)
Throttle	2 modes x 16 bits	32	

Structure	Field	Bits	Reason
L1D line	mode bit	1	which mode prefetched the line, so a useful hit is credited to that mode
L1D lines	768 lines x 1 bit	768	64 sets x 12 ways
L1D PQ and MSHR entry	mode bit	1	the mode tag travels with an in-flight prefetch
PQ + MSHR	(16 + 16) entries x 1 bit	32	
Total		29,248	about 3.6 KB

Throttling thresholds

SPP checks each mode's accuracy after every 64 of its prefetches have been filled into the L1D. Accuracy here is the fraction of those prefetched lines that a later load actually used. If accuracy is 60% or more, the mode's degree goes up one level; if it is below 40%, it goes down one level; in between, it stays the same.

We use two thresholds with a gap between them, and move only one level at a time, because a higher degree is naturally less accurate. With a single threshold the degree would keep flipping between two levels; with the gap, it only changes when accuracy is clearly high or clearly low.

We chose 60% / 40% by trying three threshold pairs, each with three top degrees, and comparing the geomean speedup over no prefetching:

Top degree	up 50% / down 30%	up 60% / down 40%	up 70% / down 50%
4	1.0145	1.0144	1.0142
6	1.0151	1.0152	1.0150
8	1.0155	1.0156	1.0152

60% / 40% with top degree 8 was best. The threshold pair matters much less than the top degree (at most 0.04% between pairs), so the result is not sensitive to the exact thresholds. The earlier single-degree version gave 1.0149.

Prefetch degrees

The levels are 1, 2, 4 and 8.

- They double, so a mode can back off quickly when it is wrong and reach a long lookahead quickly when it is right (three steps from 1 to 8).
- The top degree: 8 was better than 6 and 4 for every threshold pair (table above). The accurate mode can safely run further ahead, since the other mode has its own degree.
- The lowest level is 1, not 0, so a mode always issues its most likely next prefetch and keeps measuring its accuracy. With degree 0 it would issue nothing, never fill anything, and never get a chance to step back up.
- All four levels are used. Number of epochs at each degree in the simulated region:

Trace	**Mode**	**Degree 1**	**Degree 2**	**Degree 4**	**Degree 8**
Trace1	uniform	2	17	89	215
Trace1	pattern	121	176	166	283
Trace2	uniform	2	19	20	49
Trace2	pattern	173	2	16	191
Trace3	uniform	10	8	6	9
Trace3	pattern	104	73	81	238

IP table size

IP-stride uses 24 entries; SPP uses 64. We measured both prefetchers at both sizes (earlier SPP version, geomean speedup):

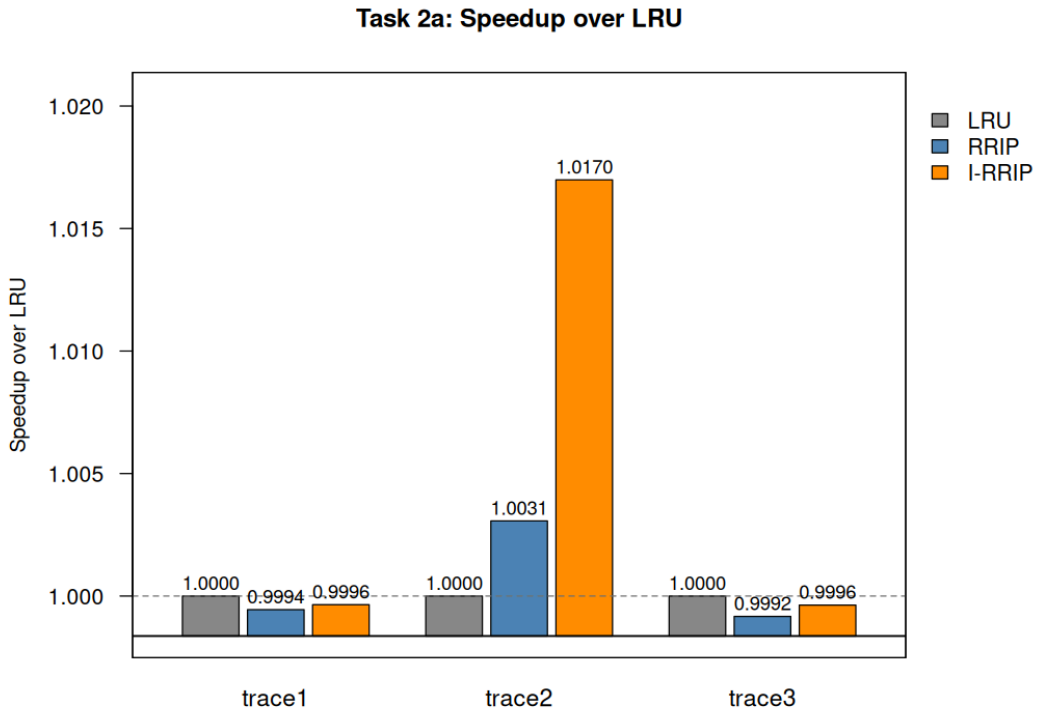
	24 entries	**64 entries**
IP-stride	1.0024	1.0054
SPP	1.0063	1.0149

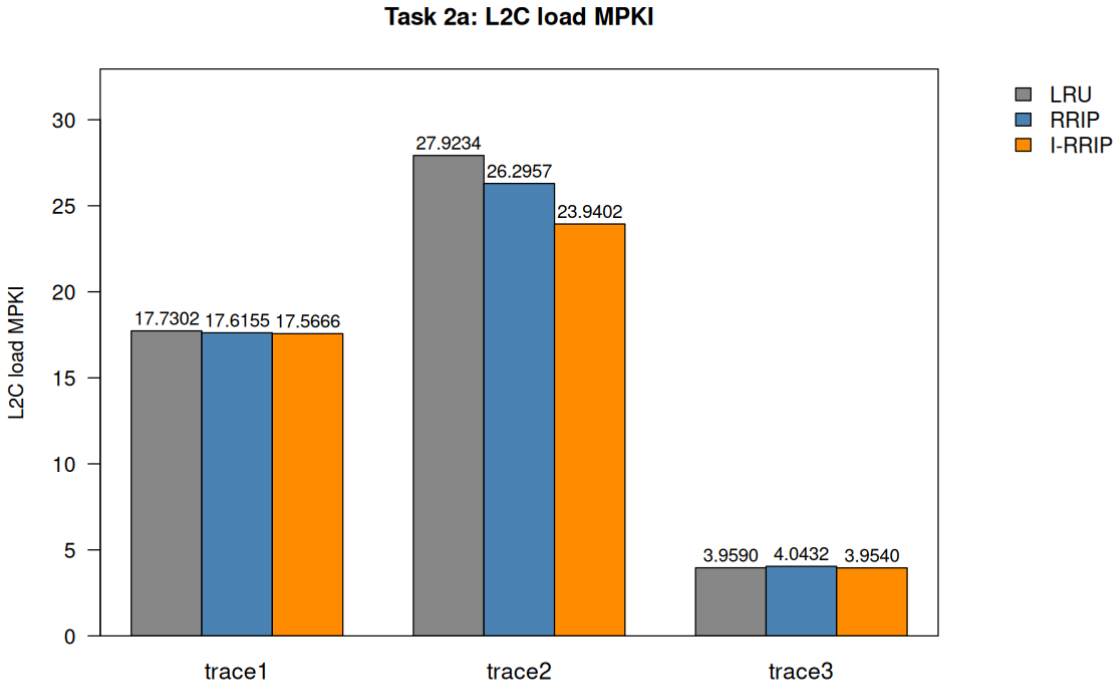
Task 2A I-RRIP

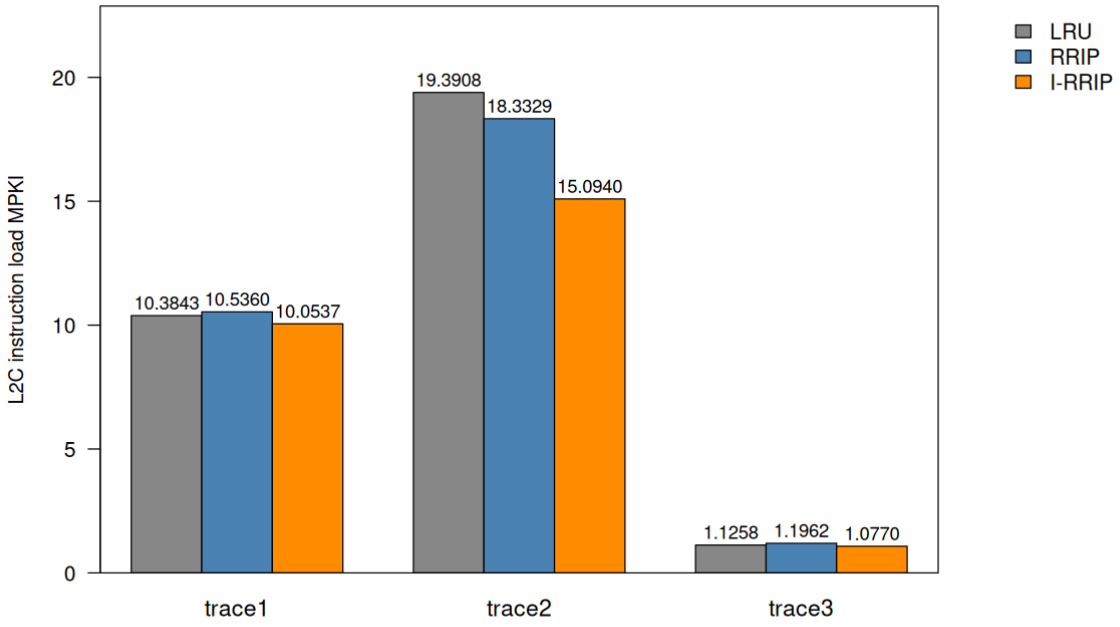
The primary goal of our policy was to reduce L2 instruction-load MPKI. However, aggressively retaining instruction lines in the L2 can create a tradeoff between instruction and data misses, so we aimed for a policy that lowers instruction misses without causing an unreasonable increase in data misses, while preferably improving L2 load behavior and IPC.

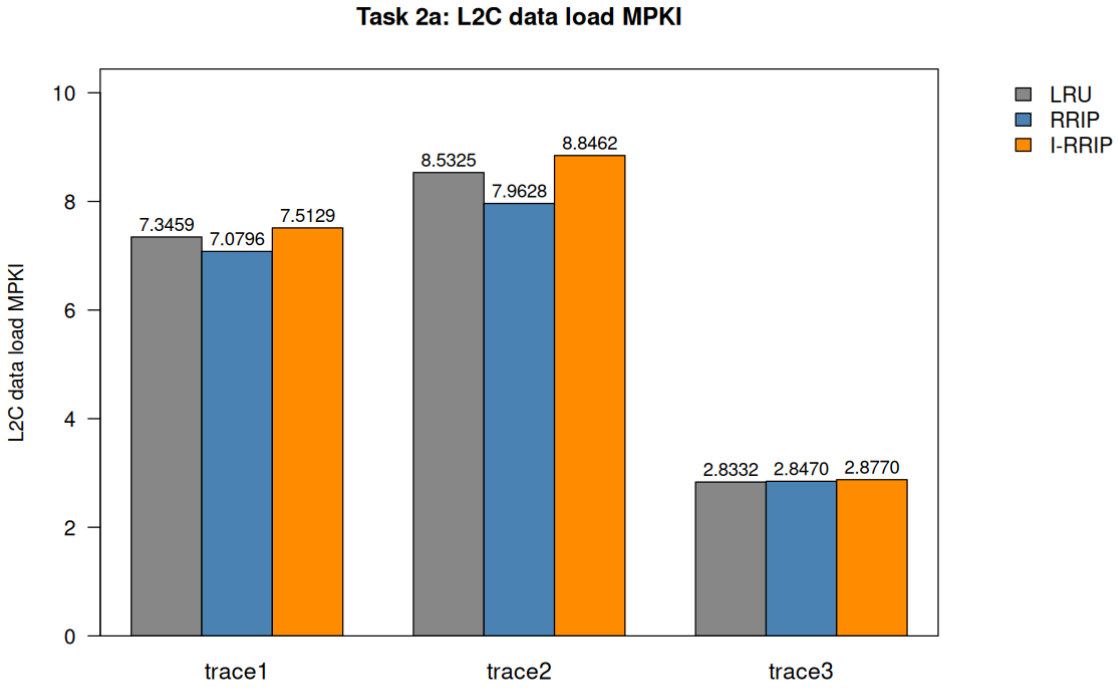
Our I-RRIP policy inserts instruction lines at RRPV 1 and non-instruction lines at RRPV 2, giving instruction lines an additional opportunity to remain in the cache. On eviction, when multiple lines have maximum RRPV, we select a non-instruction line first, thereby protecting instruction lines from being displaced by data lines. We promote both instruction and non-instruction hits to RRPV 0 because a hit is strong evidence of reuse regardless of line type; giving data hits a weaker promotion would over-prioritize instructions and could unnecessarily evict frequently reused data.

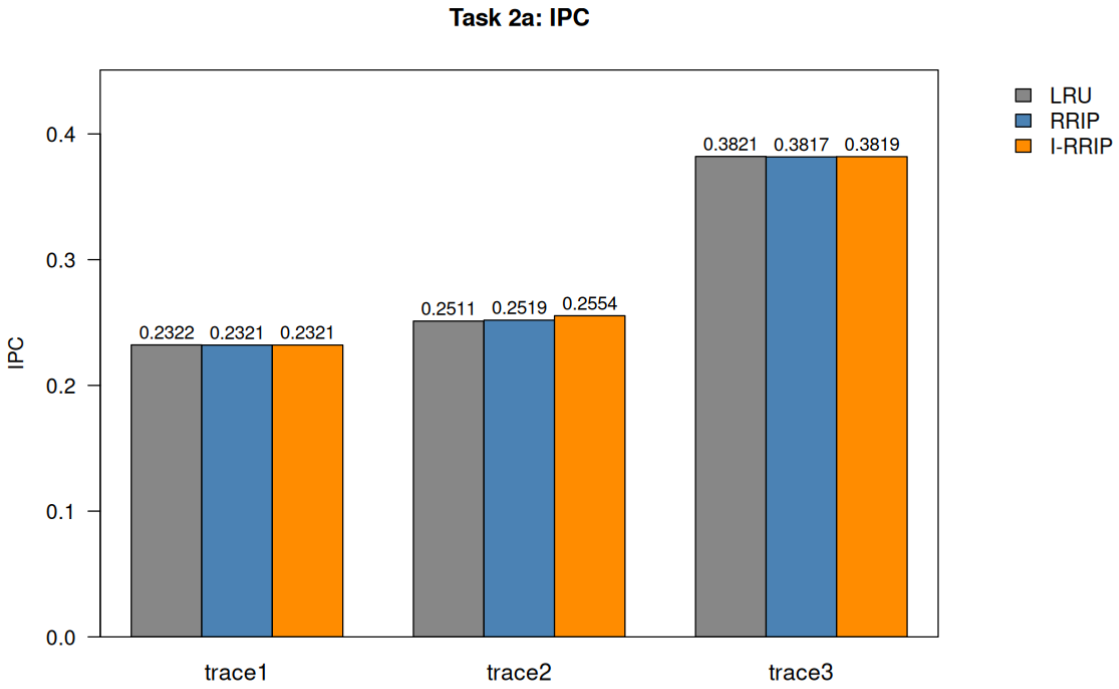
This design reduced L2 instruction-load MPKI on all three traces: from 10.536 to 10.054 for Trace1, from 18.333 to 15.094 for Trace2, and from 1.196 to 1.077 for Trace3. L2C Load MPKI also decreased on every trace, while IPC improved slightly on every trace. Although data-load MPKI increased, the increase was accepted as the cost of providing instruction awareness because the resulting instruction-miss reduction and overall L2 load behavior were favorable




Task 2a: L2C load MPKI
L2C load MPKI
0
5
10
15
20
25
30
17.7302
17.6155
17.5666
27.9234
26.2957
23.9402
3.9590
4.0432
3.9540
trace1
trace2
trace3
LRU
RRIP
I-RRIP


Task 2a: L2C instruction load MPKI
L2C instruction load MPKI
0
5
10
15
20
10.3843
10.5360
10.0537
19.3908
18.3329
15.0940
1.1258
1.1962
1.0770
trace1
trace2
trace3
LRU
RRIP
I-RRIP


Task 2a: L2C data load MPKI
L2C data load MPKI
10
8
6
4
2
0
7.3459
7.0796
7.5129
8.5325
7.9628
8.8462
2.8332
2.8470
2.8770
trace1
trace2
trace3
LRU
RRIP
I-RRIP


Task 2a: IPC
IPC
0.4
0.3
0.2
0.1
0.0
0.2322
0.2321
0.2321
0.2511
0.2519
0.2554
0.3821
0.3817
0.3819
trace1
trace2
trace3
LRU
RRIP
I-RRIP

Task 2b: Dynamic I-RRIP (set-dueling)

Setup

L2 replacement only, no prefetcher.
25M warmup + 25M simulated instructions,
GCC 7.5.
Baseline = RRIP.
Speedup = IPC / IPC(RRIP).

Implementation

replacement/dyn_irrip.l2c_repl

DESIGN

I-RRIP mode

Task 2a policy: instruction lines inserted at RRPV 1, data at RRPV 2; on eviction a data line at maxRRPV is chosen before an instruction line; hits promoted to RRPV 0.

RRIP mode

Plain SRRIP: all lines inserted at RRPV 2, first line at maxRRPV evicted, hits promoted to RRPV 0.
Leader sets: 32 of 1024 L2 sets (set % 32 == 0) always run RRIP. The other 992 follower sets use the mode chosen by the monitor.
Monitor counter: On every non-writeback access to a leader set: instr_access, instr_miss, total_access, total_miss.
Decision: score = (instr_access + instr_miss) / (total_access + total_miss); followers use I-RRIP if score > 0.25, else RRIP. Done in integers: (ia+im)*1000 > (ta+tm)*250.
Phase adaptation: Every 4096 leader accesses all four counters are halved, so the score is a moving average and old phases fade out.
Hardware cost 4 counters x 14 bits + 12-bit epoch counter = 68 bits on top of RRIP's RRPV bits. Leader sets are identified from the set index, so need no storage.

RATIONALE

Why leaders run RRIP

They measure how instruction-heavy the traffic is under a policy that doesn't favour instructions. If they ran I-RRIP, protected instruction lines would hit more, under-reporting instruction misses and biasing the score toward the current mode.
Why 32 leader sets?
~3% of the cache. Leaders never get instruction protection, so more leaders cost more when I-RRIP wins (Trace2: +1.34% dyn vs +1.39% static). 32 leaders still sample ~1 in 32 L2 accesses, so an epoch is ~130K L2 accesses: short vs program phases, long enough to average out noise. 16 would halve the sample rate; 64 would double the unprotected sets. set % 32 spreads leaders evenly over the index space.
Why accesses + misses?
An instruction stream that keeps missing counts twice as much as one that hits. That's the case where protecting instructions pays off; if instructions already hit under RRIP there is little to gain.
Why writebacks excluded?
They are not demand accesses and carry no information about instruction pressure.

Results

Trace	Policy	IPC	Speedup vs RRIP	L2C Load MPKI	L2 Instruction Load MPKI	L2 Data Load MPKI	Epochs in I-RRIP mode
Trace1	RRIP	0.23208	1.0000	17.61550	10.53600	7.07956	-
Trace1	I-RRIP	0.23212	1.0002	17.56660	10.05370	7.51292	-
Trace1	Dyn I-RRIP (0.10)	0.23212	1.0002	17.57070	10.07200	7.49868	100.0%
Trace1	Dyn I-RRIP (0.20)	0.23215	1.0003	17.56900	10.07110	7.49788	100.0%
Trace1	Dyn I-RRIP (0.25)	0.23209	1.0001	17.58150	10.08080	7.50072	100.0%

Trace1	Dyn I-RRIP (0.35)	0.23217	1.0004	17.54370	10.04270	7.50100	84.6%
Trace1	Dyn I-RRIP (0.50)	0.23208	1.0000	17.61570	10.53620	7.07952	0.0%
Trace2	RRIP	0.25190	1.0000	26.29570	18.33290	7.96284	-
Trace2	I-RRIP	0.25540	1.0139	23.94020	15.09400	8.84616	-
Trace2	Dyn I-RRIP (0.10)	0.25525	1.0133	24.00140	15.18570	8.81576	100.0%
Trace2	Dyn I-RRIP (0.20)	0.25530	1.0135	24.00760	15.18160	8.82608	95.2%
Trace2	Dyn I-RRIP (0.25)	0.25527	1.0134	24.02600	15.20670	8.81928	95.2%
Trace2	Dyn I-RRIP (0.35)	0.25507	1.0126	24.20030	15.41880	8.78148	85.7%
Trace2	Dyn I-RRIP (0.50)	0.25465	1.0109	24.47940	15.78340	8.69604	66.7%
Trace3	RRIP	0.38174	1.0000	4.04324	1.19624	2.84700	-
Trace3	I-RRIP	0.38192	1.0005	3.95396	1.07696	2.87700	-
Trace3	Dyn I-RRIP (0.10)	0.38188	1.0004	3.95884	1.08332	2.87552	100.0%

Trace3	Dyn I-RRIP (0.20)	0.38186	1.0003	3.95752	1.08292	2.87460	100.0%
Trace3	Dyn I-RRIP (0.25)	0.38188	1.0004	3.95832	1.08296	2.87536	100.0%
Trace3	Dyn I-RRIP (0.35)	0.38175	1.0000	4.01004	1.13956	2.87048	71.4%
Trace3	Dyn I-RRIP (0.50)	0.38174	1.0000	4.04324	1.19624	2.84700	0.0%

Policy

Geomean speedup vs RRIP	
RRIP	1.0000
I-RRIP	1.0048
Dyn I-RRIP (0.10)	1.0046
Dyn I-RRIP (0.20)	1.0047
Dyn I-RRIP (0.25)	1.0046
Dyn I-RRIP (0.35)	1.0043
Dyn I-RRIP (0.50)	1.0036

SENSITIVITY

(threshold = 0.10, 0.20, 0.25, 0.35, 0.50)
0.10 - 0.25: Plateau. All three traces score above 0.25 almost all the time, so followers stay in I-RRIP mode. Matches static I-RRIP minus the leader-set cost (Trace2: 1.0134 vs 1.0139).
0.35 Some epochs drop to RRIP (71-86% I-RRIP). Trace2 loses part of its gain (1.0126; L2 instr MPKI 15.21 -> 15.42). Trace3's gain disappears (instr MPKI 1.083 -> 1.140).
0.50 Too conservative. Trace1 and Trace3 never enter I-RRIP mode and behave exactly like RRIP. Trace2 spends 1/3 of epochs in RRIP and drops to 1.0109 (instr MPKI 15.78).
Trace1 I-RRIP removes 0.48 instr MPKI but adds 0.43 data MPKI, so IPC doesn't change (all differences within +/-0.04%, i.e. noise). This is where blind instruction priority stops paying off.
Chosen threshold 0.25. It sits in the middle of the flat 0.10-0.25 region, so small changes in the workload mix don't flip the decision. Recovers 96% of static I-RRIP's gain on Trace2 and

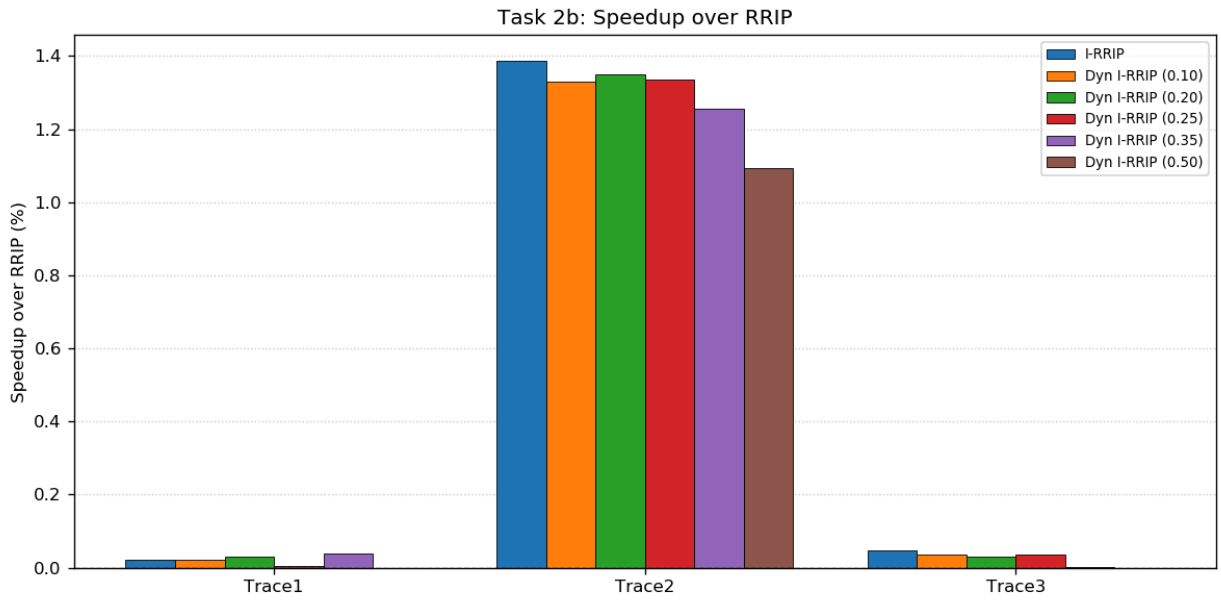
loses nothing measurable on Trace1/Trace3. Above 0.25 the policy gives up real instruction-MPKI reductions.

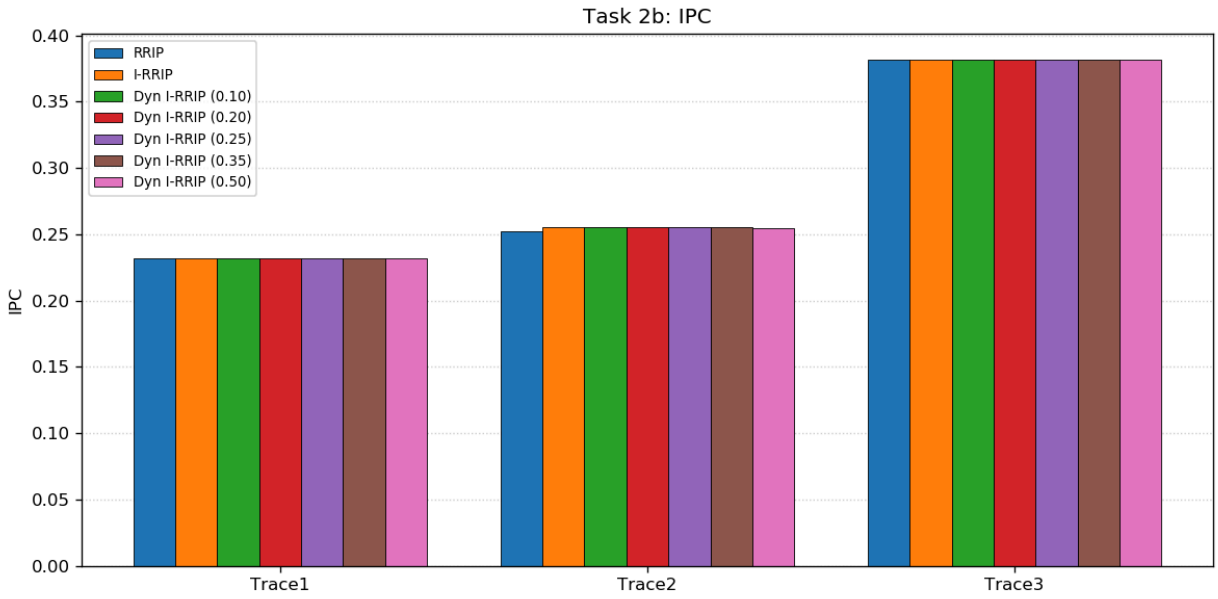
Observation

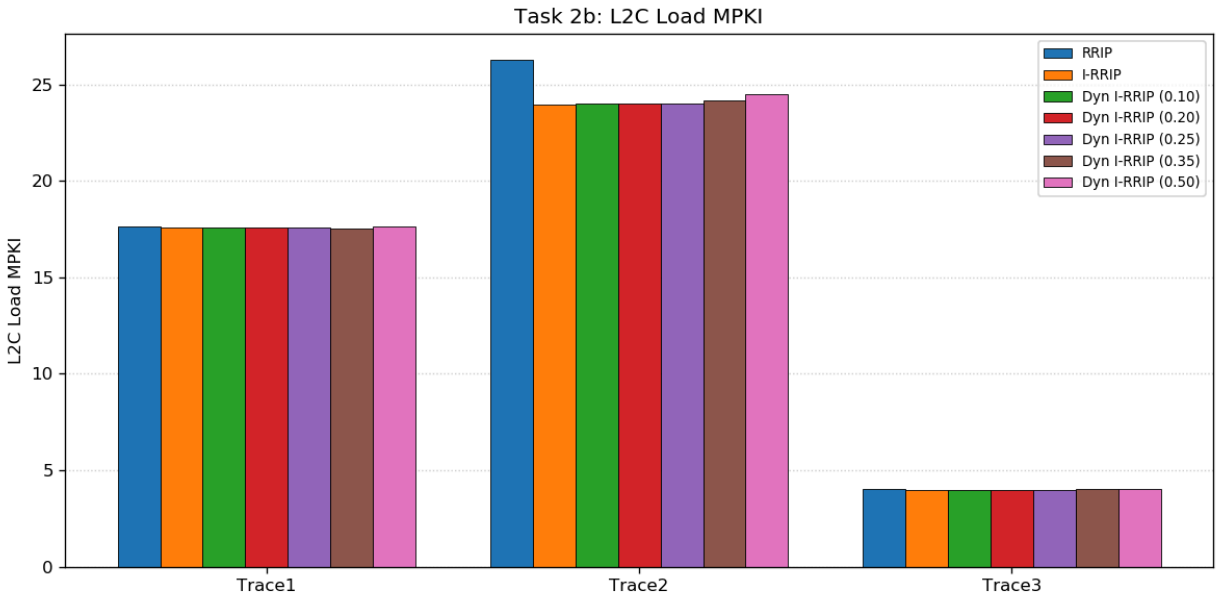
None of the traces has a long phase where favouring instructions hurts, so static I-RRIP is an upper bound here. Dyn I-RRIP gets close to it at 0.10-0.25 while staying safe: on data-heavy phases the 0.25 threshold would switch to RRIP and avoid I-RRIP's data-MPKI increase.

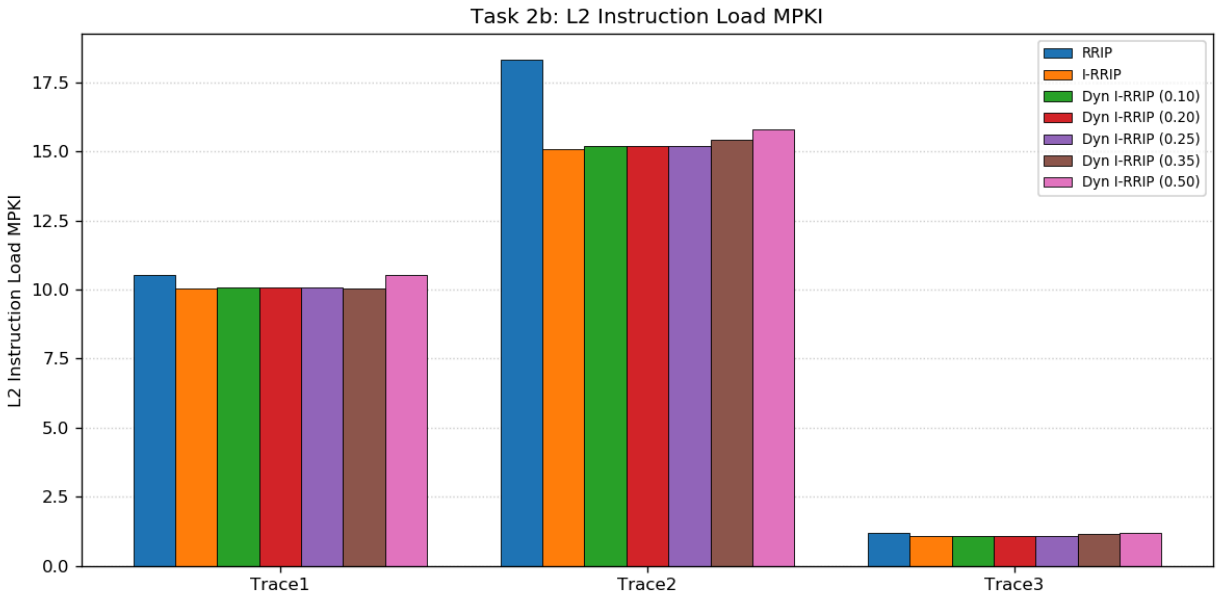
PLOTS (in docs/2b/)

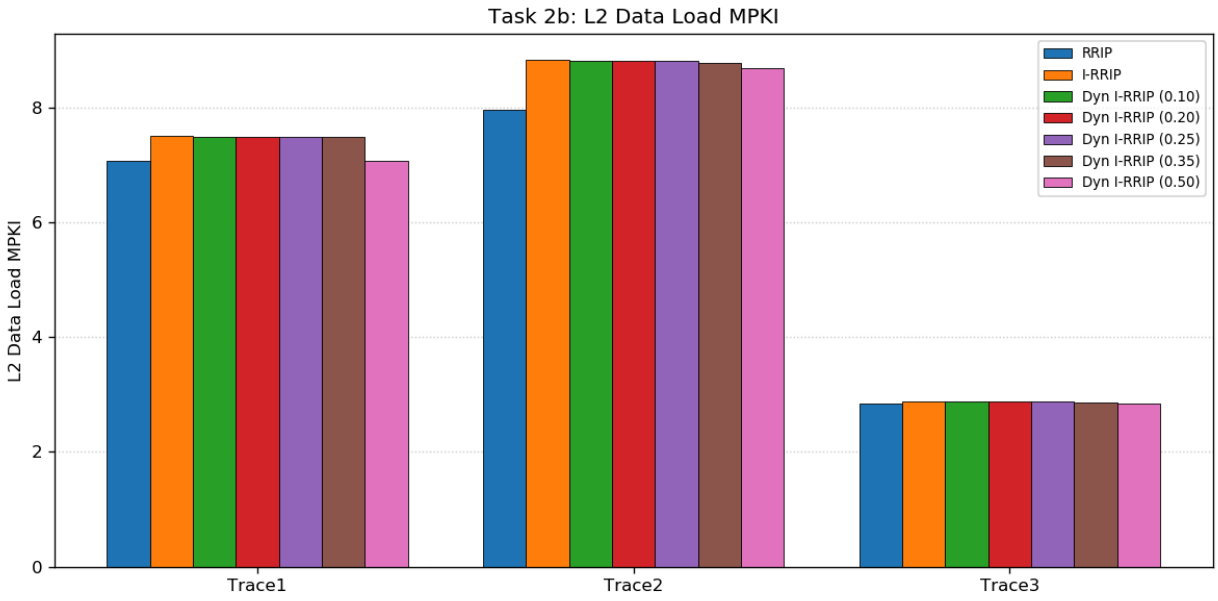
speedup.png, ipc.png, l2c-load-mpki.png, l2-instruction-load-mpki.png, l2-data-load-mpki.png, sensitivity.png

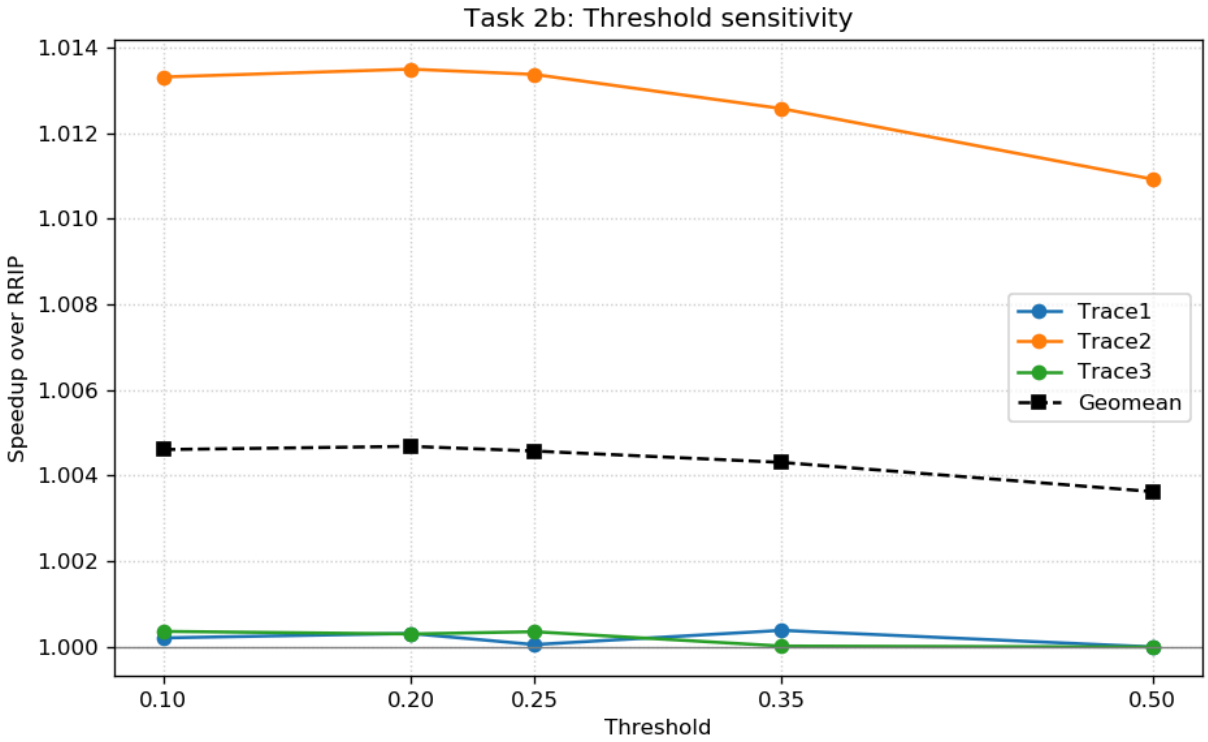



Task 2b: IPC
RRIP
I-RRIP
Dyn I-RRIP (0.10)
Dyn I-RRIP (0.20)
Dyn I-RRIP (0.25)
Dyn I-RRIP (0.35)
Dyn I-RRIP (0.50)
IPC
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40
Trace1
Trace2
Trace3


Task 2b: L2C Load MPKI
RRIP
I-RRIP
Dyn I-RRIP (0.10)
Dyn I-RRIP (0.20)
Dyn I-RRIP (0.25)
Dyn I-RRIP (0.35)
Dyn I-RRIP (0.50)
L2C Load MPKI
0
5
10
15
20
25
Trace1
Trace2
Trace3


Task 2b: L2 Instruction Load MPKI
L2 Instruction Load MPKI
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
Trace1
Trace2
Trace3
RRIP
I-RRIP
Dyn I-RRIP (0.10)
Dyn I-RRIP (0.20)
Dyn I-RRIP (0.25)
Dyn I-RRIP (0.35)
Dyn I-RRIP (0.50)


Task 2b: L2 Data Load MPKI
L2 Data Load MPKI
0
2
4
6
8
Trace1
Trace2
Trace3
RRIP
I-RRIP
Dyn I-RRIP (0.10)
Dyn I-RRIP (0.20)
Dyn I-RRIP (0.25)
Dyn I-RRIP (0.35)
Dyn I-RRIP (0.50)


Task 2b: Threshold sensitivity
Speedup over RRIP
1.014
1.012
1.010
1.008
1.006
1.004
1.002
1.000
0.10
0.20
0.25
0.35
0.50
Threshold
Trace1
Trace2
Trace3
Geomean

Task 3: Incorporating SPP with Dynamic I-RRIP

We changed dyn_irrip.l2c repl so that, in every set:

1. Prefetch requests are not counted by the leader-set monitor: they are not demand accesses, so counting them would pollute the instruction share signal that decides between I-RRIP and RRIP.
2. A prefetched line is inserted at RRPV 3, so it is the first to get evicted unless a demand uses it
3. A prefetch hit does not promote the line (only a demand hit). This is done so that repeated prefetches do not make it less likely to get evicted. It has to be demanded to get promoted.
4. Among the lines at RRPV 3, an unused prefetched line (prefetch bit still set) is evicted with higher priority.

Rules 2 and 3 are adopted from PACMan (Wu et al., MICRO 2011).

The leader sets also follow these rules.

Results

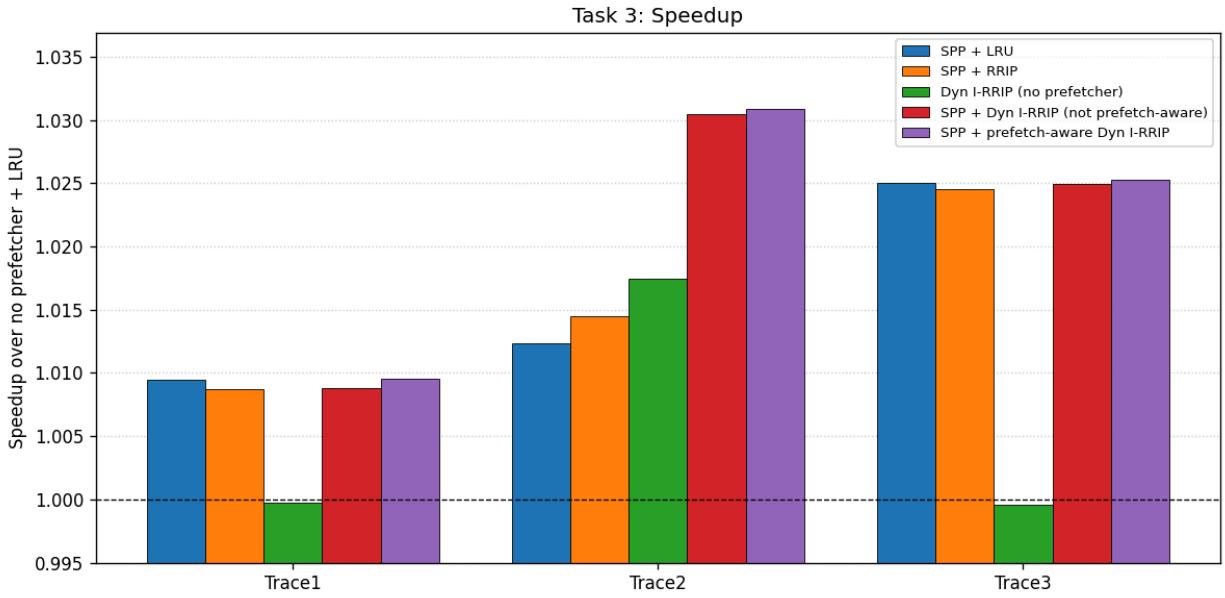
Trace	Configuration	IPC	Speedup	L1D acc.	L1D cov.	L1D load MPKI	L2C load MPKI	L2 instr. load MPKI	L2 data load MPKI
Trace1	No prefetcher + LRU	0.232345	1.0000	-	-	10.678	17.696	10.338	7.358
Trace1	SPP + LRU	0.234546	1.0095	51.3%	10.8%	9.528	16.751	10.404	6.347
Trace1	SPP + RRIP	0.234362	1.0087	51.7%	10.9%	9.519	16.747	10.645	6.102
Trace1	Dyn I-RRIP (no prefetcher)	0.232289	0.9998	-	-	10.679	17.539	10.042	7.497
Trace1	SPP + Dyn I-RRIP (not prefetch-aware)	0.234381	1.0088	51.9%	10.7%	9.531	16.596	10.107	6.489
Trace1	**SPP + prefetch-aware Dyn I-RRIP**	**0.234555**	**1.0095**	**51.4%**	**10.8%**	**9.525**	**16.487**	**10.040**	**6.447**
Trace2	No prefetcher + LRU	0.251158	1.0000	-	-	13.538	27.357	18.853	8.504
Trace2	SPP + LRU	0.254259	1.0123	51.1%	3.8%	13.018	26.898	18.894	8.004
Trace2	SPP + RRIP	0.254800	1.0145	51.2%	3.7%	13.031	25.271	17.809	7.462
Trace2	Dyn I-RRIP (no prefetcher)	0.255532	1.0174	-	-	13.540	23.219	14.387	8.833
Trace2	SPP + Dyn I-RRIP (not prefetch-aware)	0.258805	1.0304	51.1%	3.8%	13.022	22.706	14.391	8.315
Trace2	**SPP + prefetch-aware Dyn I-RRIP**	**0.258921**	**1.0309**	**51.4%**	**3.8%**	**13.021**	**22.550**	**14.240**	**8.310**
Trace3	No prefetcher + LRU	0.382065	1.0000	-	-	3.403	3.954	1.124	2.830
Trace3	SPP + LRU	0.391621	1.0250	48.0%	21.5%	2.672	3.089	1.142	1.947
Trace3	SPP + RRIP	0.391432	1.0245	48.3%	21.2%	2.682	3.174	1.204	1.970
Trace3	Dyn I-RRIP (no prefetcher)	0.381908	0.9996	-	-	3.403	3.957	1.082	2.875
Trace3	SPP + Dyn I-RRIP (not prefetch-aware)	0.391587	1.0249	47.6%	21.5%	2.671	3.090	1.099	1.991
Trace3	**SPP + prefetch-aware Dyn I-RRIP**	**0.391726**	**1.0253**	**48.0%**	**21.6%**	**2.667**	**3.048**	**1.070**	**1.978**

Speedup of SPP + Dyn-IRRP Prefetch aware

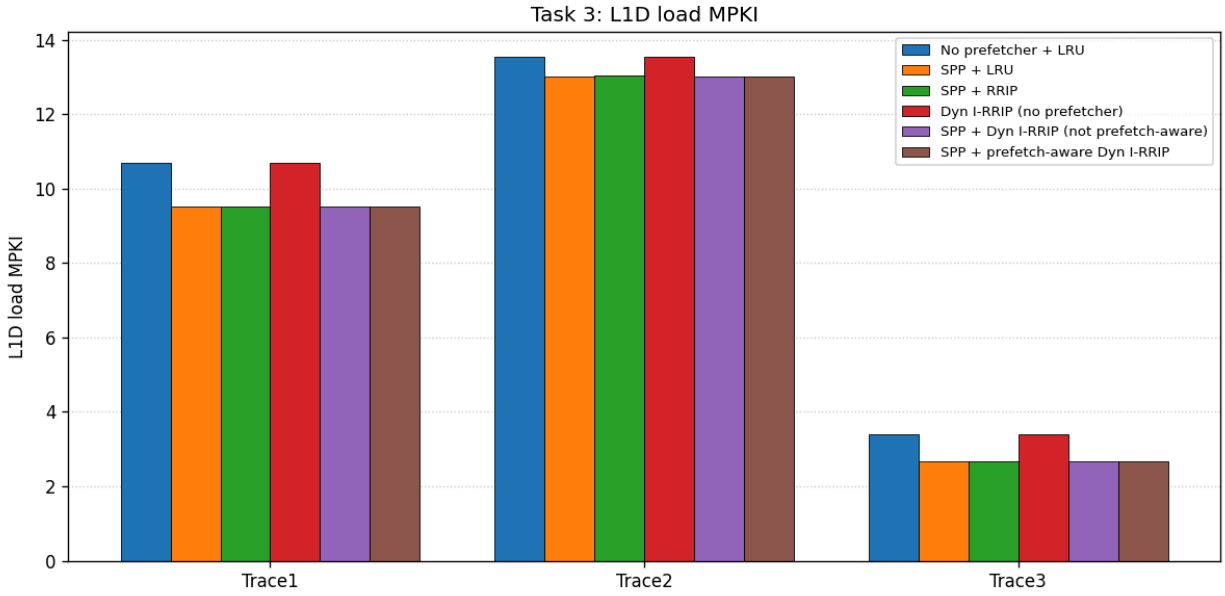
Reference	Trace1	Trace2	Trace3	Geomean
No prefetcher + LRU	1.0095	1.0309	1.0253	1.0219
SPP + LRU	1.0000	1.0183	1.0003	1.0062
SPP + RRIP	1.0008	1.0162	1.0008	1.0059
Dyn I-RRIP (no prefetcher)	1.0098	1.0133	1.0257	1.0162
SPP + Dyn I-RRIP (not prefetch-aware)	1.0007	1.0004	1.0004	1.0005

Configuration	Geomean speedup over no prefetcher
No prefetcher + LRU	1.0000
SPP + LRU	1.0156
SPP + RRIP	1.0159
Dyn I-RRIP (no prefetcher)	1.0056
SPP + Dyn I-RRIP (not prefetch-aware)	1.0213
SPP + prefetch-aware Dyn I-RRIP	**1.0219**

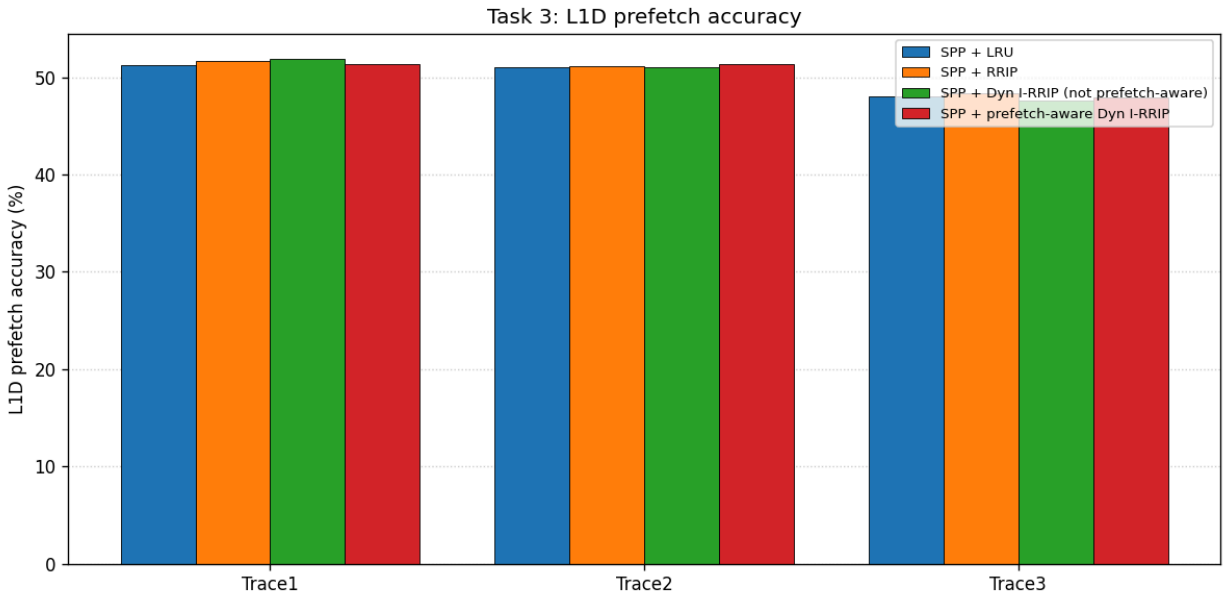
Plots



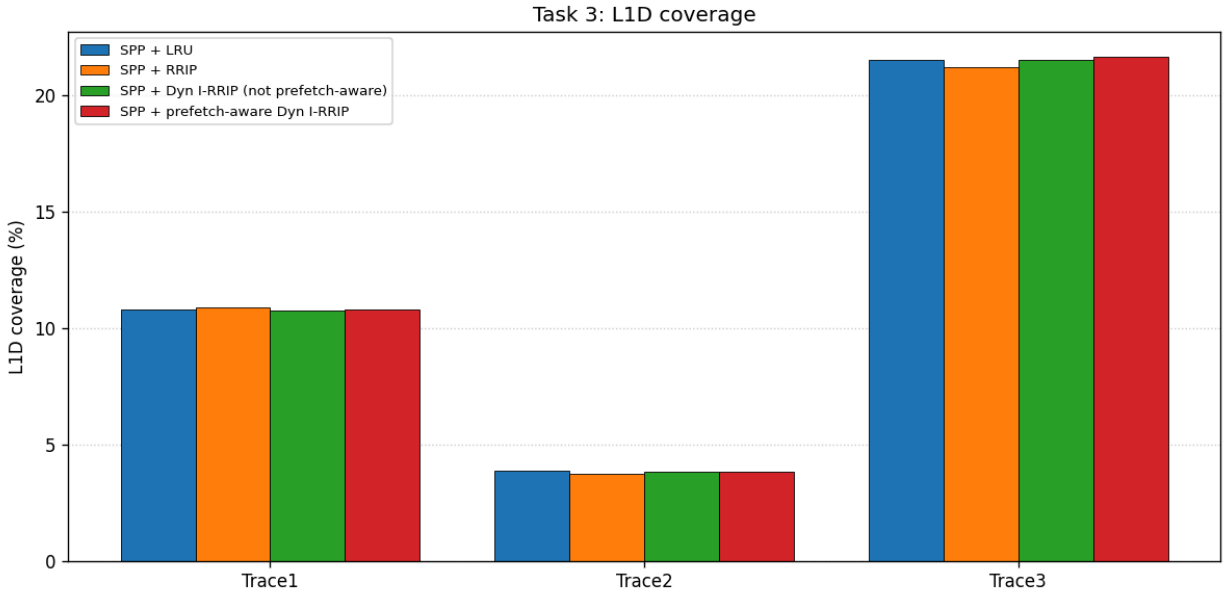
Speedup over no prefetcher + LRU. SPP + prefetch-aware Dyn I-RRIP has the highest IPC on all three traces (1.0219 geomean).



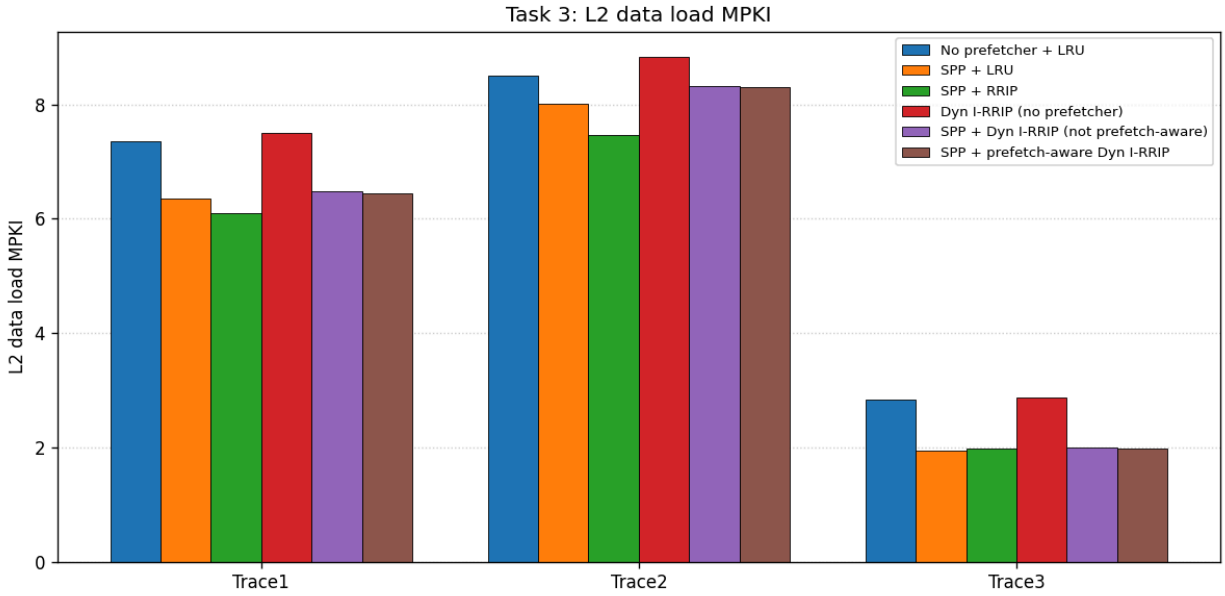
SPP lowers L1D load MPKI on every trace; the L2 policy has almost no effect on it

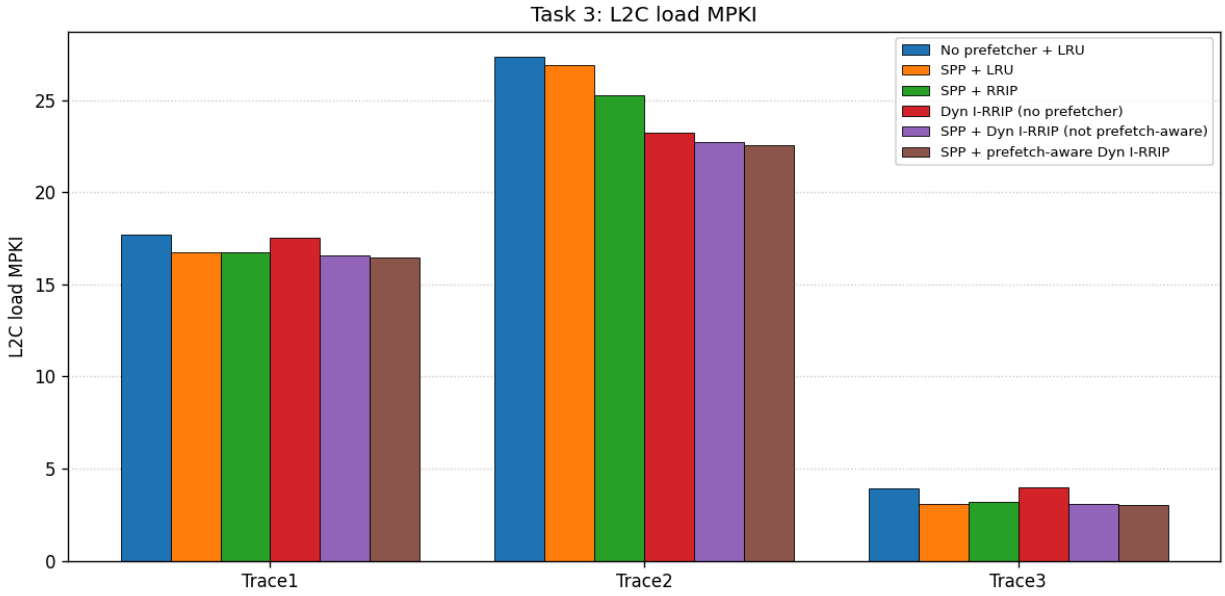


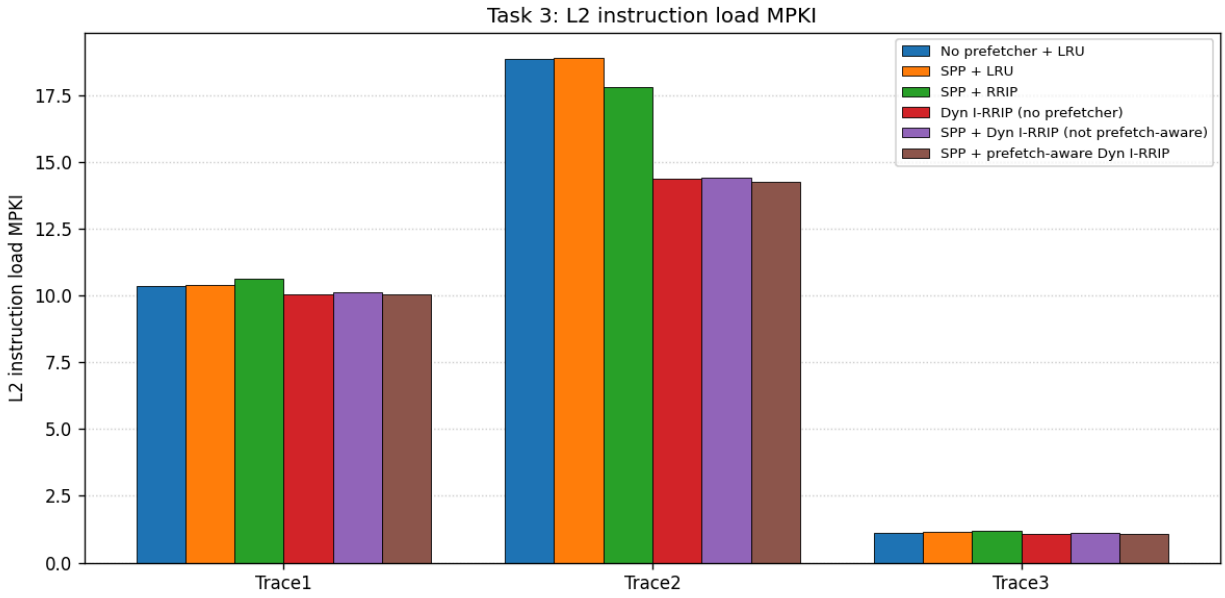
Accuracy stays at 48–52% for all SPP configurations. The L2 policy should not directly affect prefetch accuracy.



Coverage is about the same for all SPP configurations




Task 3: L2C load MPKI
L2C load MPKI
0
5
10
15
20
25
Trace1
Trace2
Trace3
No prefetcher + LRU
SPP + LRU
SPP + RRIP
Dyn I-RRIP (no prefetcher)
SPP + Dyn I-RRIP (not prefetch-aware)
SPP + prefetch-aware Dyn I-RRIP


Task 3: L2 instruction load MPKI
L2 instruction load MPKI
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
Trace1
Trace2
Trace3
No prefetcher + LRU
SPP + LRU
SPP + RRIP
Dyn I-RRIP (no prefetcher)
SPP + Dyn I-RRIP (not prefetch-aware)
SPP + prefetch-aware Dyn I-RRIP